JAVA STREAM API & LAMBDA EXPRESSIONS

Tài Liệu Ôn Tập Vấn Đáp Toàn Diện — Đồ Án PRO192 (Library Management System)

Dành cho các thành viên Nhóm 01 • PRO192 Fall 2026

📌 **Khái niệm cốt lõi:** Stream API (Java 8+) là cơ chế xử lý dữ liệu tập hợp (Collections) theo phong cách khai báo (Declarative Pipeline) giống như một băng chuyền nhà máy. Dữ liệu chạy qua các trạm trung gian (lọc, biến đổi, sắp xếp) và được đóng gói ở trạm kết thúc mà không làm thay đổi dữ liệu gốc.

1. Băng Chuyền Dữ Liệu 3 Giai Đoạn (Stream Pipeline Architecture)

```
[ 1. NGUỒN (Source) ]          -->  memberList.stream()
          |
          ▼
[ 2. TRUNG GIAN (Intermediate) ] -->  .filter(...)  --> .map(...)  --> .sorted(...)  (LAZY: Chưa chạy ngay)
          |
          ▼
[ 3. KẾT THÚC (Terminal) ]       -->  .collect(Collectors.toList()) / .sum()        (EAGER: Chạy & Đóng Stream)
```

2. Bảng Tra Cứu Các Thao Tác Trung Gian (Intermediate Operations - Lazy)

Đặc tính: Luôn trả về một `Stream<T>` mới. Không thực thi tính toán cho tới khi gặp Terminal Operation.

Phương thức	Ý nghĩa & Bản chất	Vị trí trong dự án	Ví dụ thực tế trong Code
`filter(Predicate)`	Lọc phần tử. Giữ lại phần tử nếu biểu thức Lambda trả về `true`.	`MemberDAO` `BookDAO`	`.filter(m -> m.getMembershipTier().equals("VIP"))` *Lọc độc giả VIP*
`map(Function)`	**Ánh xạ (Transform):** Rút trích hoặc biến đổi đối tượng sang kiểu dữ liệu khác.	`BorrowRecord` `BookDAO`	`.map(Book::getBookId)` *Biến Stream<Book> thành Stream<String>*
`mapToDouble(Function)`	Biến đổi đối tượng thành luồng số thực nguyên thủy `DoubleStream` để tính toán.	`BorrowDAO` `BorrowRecord`	`.mapToDouble(b -> b.calculateBorrowFee(days))` *Lấy phí mượn từng cuốn sách*
`mapToInt(Function)`	Biến đổi thành `IntStream` giúp tiết kiệm RAM (tránh boxing/unboxing).	`Program.java` `BookDAO`	`.mapToInt(Book::getQuantity)` *Lấy số lượng tồn kho*
`sorted(Comparator)`	Sắp xếp luồng theo tiêu chuẩn truyền vào (dùng `Comparator.comparing`).	`MemberDAO` `BookDAO`	`.sorted(Comparator.comparing(Member::getReputationPoints).reversed())`
`distinct()`	Loại bỏ các phần tử trùng lặp (dựa trên `equals()` và `hashCode()`).	`BorrowDAO`	`.map(Book::getAuthor).distinct()` *Lấy danh sách tác giả không trùng*
`limit(n)`	Cắt luồng, chỉ lấy tối đa `n` phần tử đầu tiên (Short-circuiting).	`BookDAO`	`.limit(3)` *Gợi ý Top 3 sách tương tự*
`flatMap(Function)`	**Làm phẳng:** Gộp nhiều List/Stream con lồng nhau thành 1 Stream phẳng duy nhất.	`BorrowDAO`	`.flatMap(r -> r.getBorrowedBooks().stream())` *Gộp toàn bộ sách của mọi phiếu mượn*

3. Bảng Tra Cứu Các Thao Tác Kết Thúc (Terminal Operations - Eager)

Đặc tính: Kích hoạt toàn bộ pipeline chạy, thu hoạch kết quả và đóng luồng Stream.

Phương thức	Ý nghĩa & Bản chất	Vị trí trong dự án	Ví dụ thực tế trong Code
`collect(Collector)`	Thu hoạch kết quả đóng gói vào `List`, `Set`, hoặc nối chuỗi.	Toàn bộ dự án	`.collect(Collectors.toList())` `.collect(Collectors.joining(","))`
`forEach(Consumer)`	Duyệt qua từng phần tử và thực thi hành động (không trả về dữ liệu).	`BookManagement` `BorrowManagement`	`lowStockBooks.forEach(b -> System.out.println(b.getTitle()))`
`sum()`	Tính tổng toàn bộ các số trong luồng `DoubleStream` hoặc `IntStream`.	`Program.java` `BorrowDAO`	`.mapToDouble(BorrowRecord::getFineFee).sum()` *Tổng doanh thu tiền phạt*
`count()`	Đếm số lượng phần tử còn lại trên luồng (trả về kiểu `long`).	`BookDAO` `BorrowDAO`	`.filter(b -> b.getQuantity() == 0).count()` *Đếm số sách đã hết hàng*
`findFirst()`	Lấy phần tử hợp lệ đầu tiên trên luồng (trả về `Optional<T>`).	`BookDAO` `MemberDAO`	`.filter(b -> b.getBookId().equals(id)).findFirst()`
`orElse(defaultValue)`	Dùng kèm với `findFirst()`: Trả về đối tượng nếu tìm thấy, hoặc giá trị mặc định.	`BookDAO` `MemberDAO`	`.findFirst().orElse(null)` *Trả về null nếu không tìm thấy ID*
`anyMatch(Predicate)`	Trả về `true` nếu có **ít nhất 1 phần tử** thỏa mãn điều kiện.	`MemberDAO`	`.anyMatch(id -> !ids.add(id))` *Phát hiện trùng lặp mã độc giả*
`allMatch(Predicate)`	Trả về `true` nếu **tất cả các phần tử** đều thỏa mãn điều kiện.	`BorrowDAO`	`.allMatch(b -> b.getQuantity() > 0)` *Kiểm tra tất cả sách mượn đều còn hàng*
`noneMatch(Predicate)`	Trả về `true` nếu **không có phần tử nào** thỏa mãn điều kiện.	`BorrowDAO`	`.noneMatch(r -> r.getRecordId().equals(newId))`
`max(Comparator)`	Tìm phần tử lớn nhất theo tiêu chuẩn so sánh (trả về `Optional<T>`).	`Menu.java`	`values.stream().max(Integer::compareTo).orElse(0)` *Tìm tồn kho lớn nhất vẽ biểu đồ*

4. Minh Họa Đối Chiếu Trực Quan: Cách Cũ vs Stream API

Bài toán: Lọc độc giả VIP (Điểm >= 100), sắp xếp họ tên tăng dần, và lấy danh sách Tên:

```
// CÁCH CŨ (23 dòng - Vòng lặp for & Anonymous Class):
List<Member> vip = new ArrayList<>();
for (Member m : memberList) {
    if (m.getReputationPoints() >= 100) vip.add(m);
}
Collections.sort(vip, new Comparator<Member>() {
    public int compare(Member m1, Member m2) {
        return m1.getFullName().compareTo(m2.getFullName());
    }
});
List<String> names = new ArrayList<>();
for (Member m : vip) names.add(m.getFullName());

// CÁCH MỚI (Stream API + Lambda - 5 dòng súc tích):
return memberList.stream()
        .filter(m -> m.getReputationPoints() >= 100)    // 1. Lọc điểm >= 100
        .map(Member::getFullName)                       // 2. Ánh xạ lấy Tên trước (Tối ưu JVM!)
        .sorted()                                       // 3. Sắp xếp thứ tự tự nhiên của String
        .collect(Collectors.toList());                  // 4. Thu hoạch thành List
```

> 💡 **Mẹo Trả Lời Vấn Đáp Điểm 10:**
> Khi thầy hỏi tại sao lại đảo `.map()` lên trước `.sorted()` ? Trả lời: "*Dạ thưa thầy, đưa `.map()` lên trước giúp trích xuất chuỗi String ngay từ đầu. Hàm*

> *getter chỉ phải gọi đúng N lần. Sau đó thuật toán sắp xếp (TimSort) chỉ cần so sánh trực tiếp thứ tự tự nhiên của String, tránh việc liên tục gọi lại getter qua Comparator hàng trăm lần trong quá trình sort, giúp tối ưu hiệu năng JVM ạ!"*
